Iac Provisioning for Real Estate System

Course Name:

Devops

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	CHERRY MEHTA	EN22CS301292
2	BHUMIKA CHOYAL	EN22CS301276
3	AYUSH JAIN	EN22CS301247
4	AYUSH PATIDAR	EN22CS301253
5	DIYA GOYAL	EN22CS301351

Group Name: 03D3

Project Number: DO-27

Industry Mentor Name: Mr. Vaibhav

University Mentor Name: Prof. Shyam Patel

Academic Year: 2026

Table of Contents

Problem Statement & Objectives

1. Problem Statement
2. Project Objectives
3. Scope of the Project

Proposed Solution

1. Key features
2. Overall Architecture / Workflow
3. Tools & Technologies Used

Results & Output

1. Screenshots / outputs
2. Reports / dashboards / models
3. Key outcomes

Conclusion

Future Scope & Enhancements

Problem Statement & Objective

1.Problem Statement

In modern real estate applications, deploying and managing infrastructure manually is a complex, time-consuming, and error-prone process. Traditional methods require system administrators to manually configure servers, set up networking, install dependencies, and deploy applications, which often leads to inconsistencies across environments and delays in delivery.
To overcome these challenges, there is a need for an automated system that can provision infrastructure, configure environments, and deploy applications efficiently with minimal human intervention.

2.Project Objectives

The primary objective of this project is to design and implement an automated Infrastructure as Code (IaC) based system for efficient provisioning, configuration, and deployment of a real estate application using DevOps practices.

- To automate infrastructure provisioning using Terraform.
- To create and manage AWS resources such as EC2, VPC, and S3.
- To implement configuration management using Ansible.
- To automate server setup and software installation.
- To containerize the application using Docker.
- To ensure application portability across different environments.
- To build and integrate CI/CD pipelines using GitHub Actions or Jenkins.
- To automate build, test, and deployment processes.

3.Scope of the Project

The scope of this project is to design and implement an automated Infrastructure as Code (IaC) based system for provisioning, configuring, and deploying a real estate application using DevOps practices. The project focuses on improving efficiency, consistency, and scalability of infrastructure and application deployment

In-Scope:

- **Cloud Deployment on AWS**: Deploy and manage infrastructure on AWS for scalable and reliable hosting.
- **Monitoring and Logging**: Track system performance, logs, and failures for better system reliability.
- **Security Implementation**:Apply IAM roles, encryption, and secure communication protocols.
- **Scalability and High Availability**:Design the system to handle increasing user load using auto-scaling and load balancing.
- **Multi-Environment Support**: Support deployment in both cloud environments and local containerized setup

Proposed Solution

1. **Key Features**

- **Infrastructure as Code (IaC):-** Automated infrastructure provisioning using Terraform.
- **Automated Configuration Management:-**Server setup and dependency installation using Ansible.
- **Containerized Application Deployment:-**Application packaging and deployment using Docker.
- **CI/CD Pipeline Automation:-** Continuous integration and deployment using GitHub Actions or Jenkins.
- **Cloud-Based Deployment (AWS):-**Scalable and reliable hosting on AWS infrastructure.
- **Automated End-to-End Workflow:-**Full automation from code commit to application deployment.
- **Scalability Support:-** Ability to handle increased load using auto-scaling mechanisms.

2. **Overall Architecture / Workflow**

The proposed system follows a **layered and modular DevOps architecture**, where each component performs a specific role in the deployment lifecycle. The architecture ensures automation, scalability, and consistency across all environments.

Architecture Layers:

1. **Version Control Layer**
    - Source code, infrastructure code, and configuration files are stored in Git repositories.
    - Enables versioning, collaboration, and change tracking.

2. **CI/CD Layer**
   - Tools like GitHub Actions or Jenkins automate build, test, and deployment processes.
   - Triggers pipelines on every code commit.
3. **Infrastructure Layer (IaC)**
   - Terraform is used to provision cloud infrastructure such as EC2, VPC, IAM, and S3 on AWS.
   - Ensures consistent and repeatable infrastructure setup.
4. **Configuration Layer**
   - Ansible configures servers and installs required dependencies.
   - Ensures idempotent and automated configuration.
5. **Containerization Layer**
   - Docker is used to package the application into containers.
   - Provides portability and environment consistency.
6. **Execution Layer (Cloud/Local)**
   - Application is deployed on AWS cloud or local Docker environments.
   - Ensures scalability and availability.
7. **Monitoring and Logging Layer**
   - Tracks system performance, logs, and failures.
   - Helps in maintaining reliability and quick issue resolution.

Workflow:

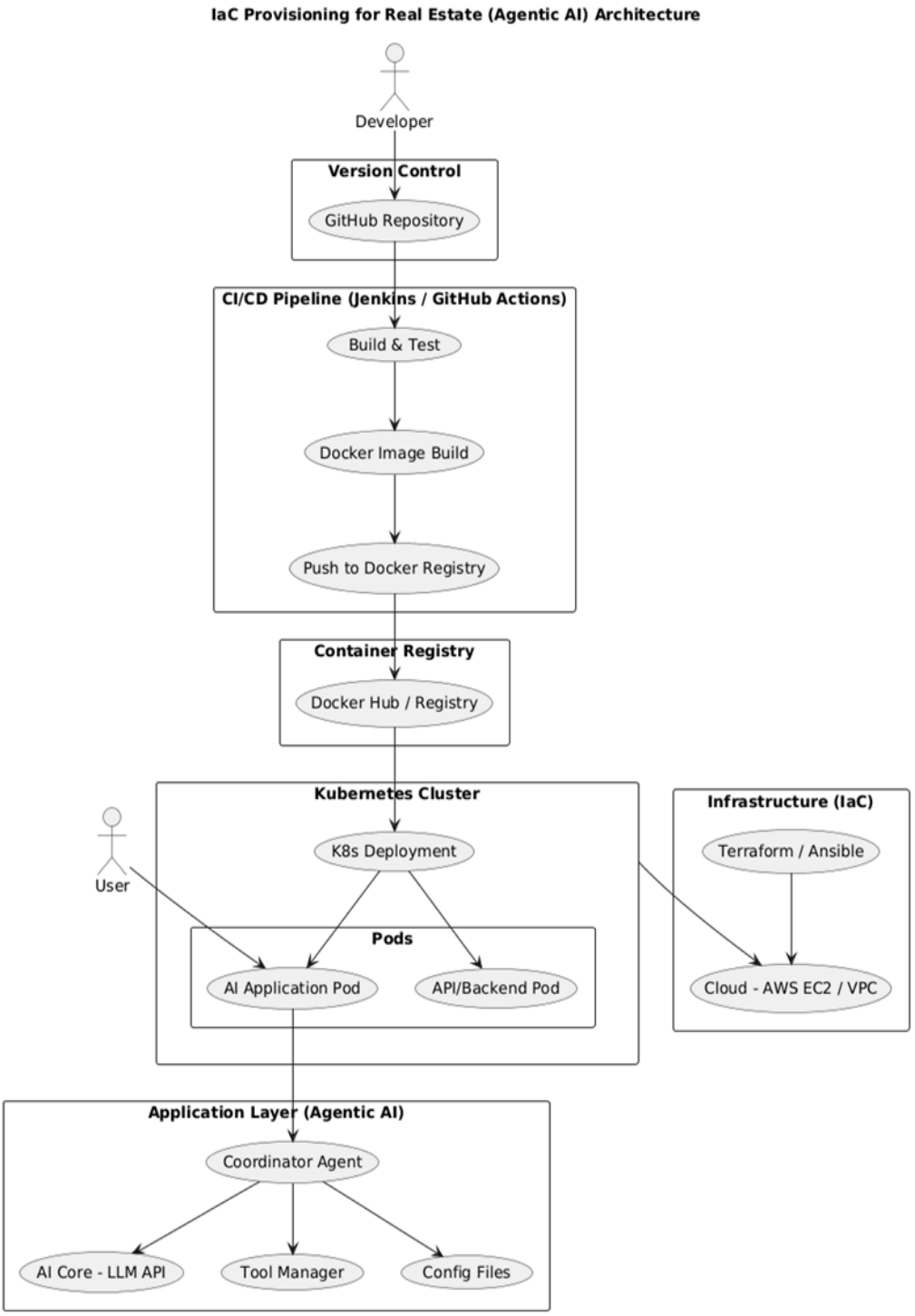
The workflow defines the step-by-step process from code development to deployment:

- Developer writes code and pushes it to the Git repository.
- CI/CD pipeline is automatically triggered.
- The pipeline validates code and builds a Docker image.
- Terraform provisions infrastructure on AWS.
- Cloud resources such as EC2 instances and networking are created.
- Ansible connects to instances via SSH and configures them.
- Required software like Docker is installed on the servers.
- The Docker container is deployed on the provisioned infrastructure.
- The application becomes live and accessible to users.
- Monitoring tools continuously track logs and system performance.

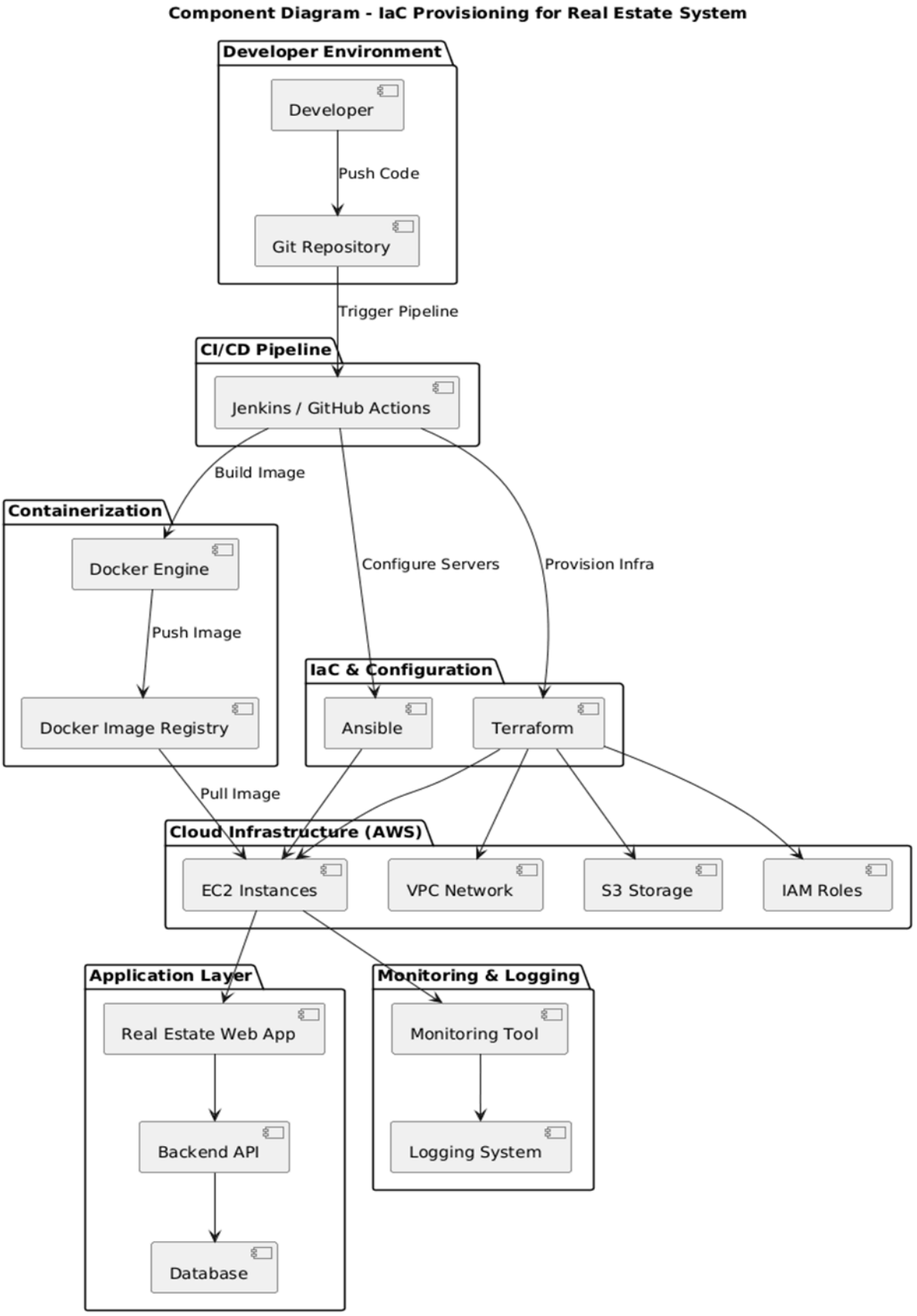
System Architecture

The system architecture is designed using a layered and modular DevOps approach to automate infrastructure provisioning, configuration, and deployment of a real estate

application. Each layer is responsible for a specific function and integrates seamlessly with other components to ensure efficiency, scalability, and reliability.



Architecture Diagram



Component Diagram

Tools and Technologies Used

The project utilizes a combination of DevOps tools and technologies to automate infrastructure provisioning, configuration, and deployment of the real estate application.

The major tools and technologies used in this project are explained below in detailed:

1. Infrastructure as Code (IaC)

- **Tool:** Terraform
- **Purpose:** Automates provisioning of cloud infrastructure such as EC2, VPC, IAM, and S3 on AWS.
- **Language Used:** HCL (HashiCorp Configuration Language)

2. Configuration Management

- **Tool:** Ansible
- **Purpose:** Automates server configuration, software installation, and environment setup.
- **Language Used:** YAML

3. Containerization

- **Tool:** Docker
- **Purpose:** Packages the application and its dependencies into containers for consistency and portability.
- **Language Used:** Dockerfile syntax

4. CI/CD Pipeline

- **Tools:** GitHub Actions / Jenkins
- **Purpose:** Automates build, test, and deployment processes.
- **Language Used:** YAML (GitHub Actions), Groovy (Jenkins)

5. Cloud Platform

- **Platform:** Amazon Web Services
- **Services Used:** EC2, S3, VPC, IAM
- **Purpose:** Provides scalable and reliable cloud infrastructure.

6. Version Control System

- **Tool:** Git
- **Purpose:** Manages source code, infrastructure code, and configuration files.

7. Programming Language

- **Language:** JavaScript (Node.js) / Python (optional)
- **Purpose:** Used for developing the real estate application backend.

8. Monitoring and Logging

- **Tools:** AWS CloudWatch / Logging tools
- **Purpose:** Tracks system performance, logs, and failures.

9. Operating System

- **Platform:** Linux (Ubuntu)
- **Purpose:** Used as the base environment for deployment and server configuration

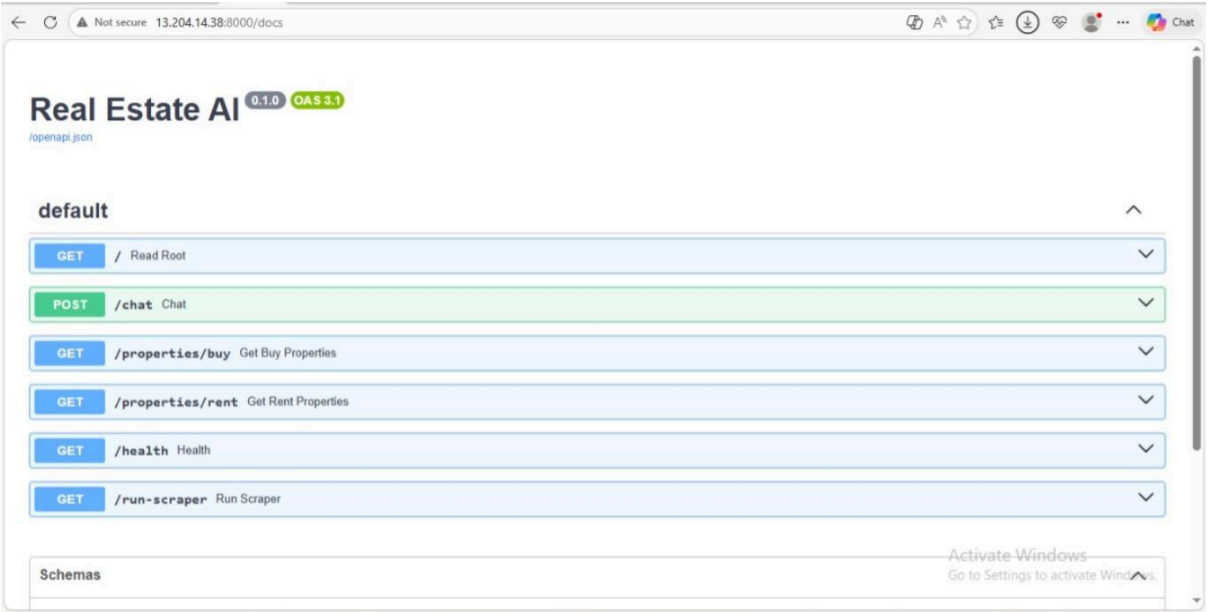
Results & Output

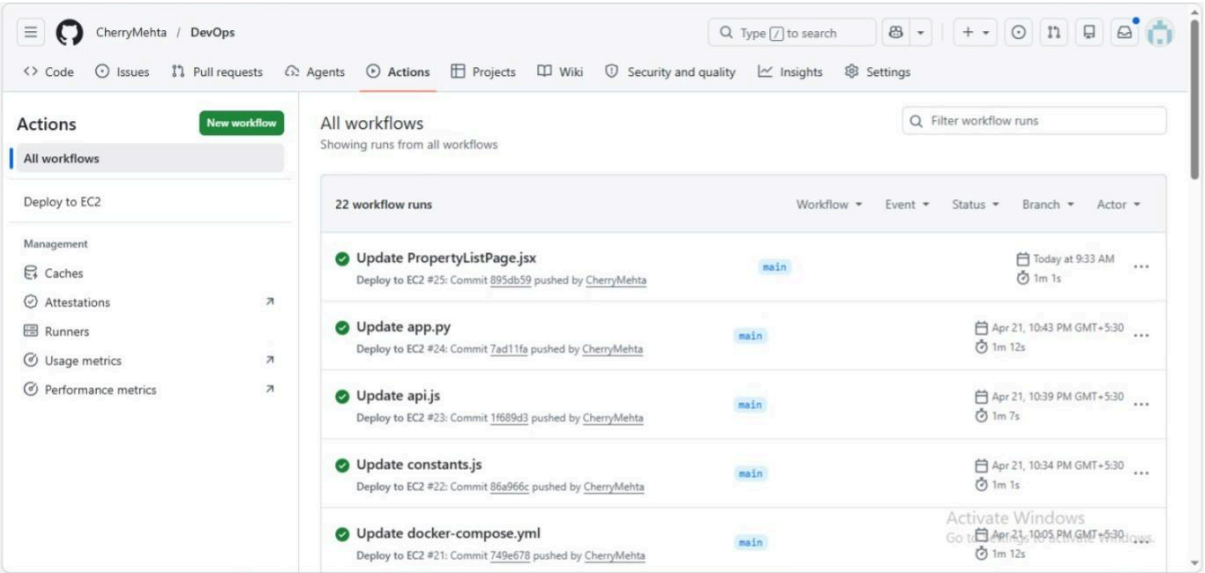
The implementation of the Infrastructure as Code (IaC) based DevOps system successfully achieved automated provisioning, configuration, and deployment of the real estate application. The system demonstrates improved efficiency, consistency, and scalability compared to traditional manual deployment methods.

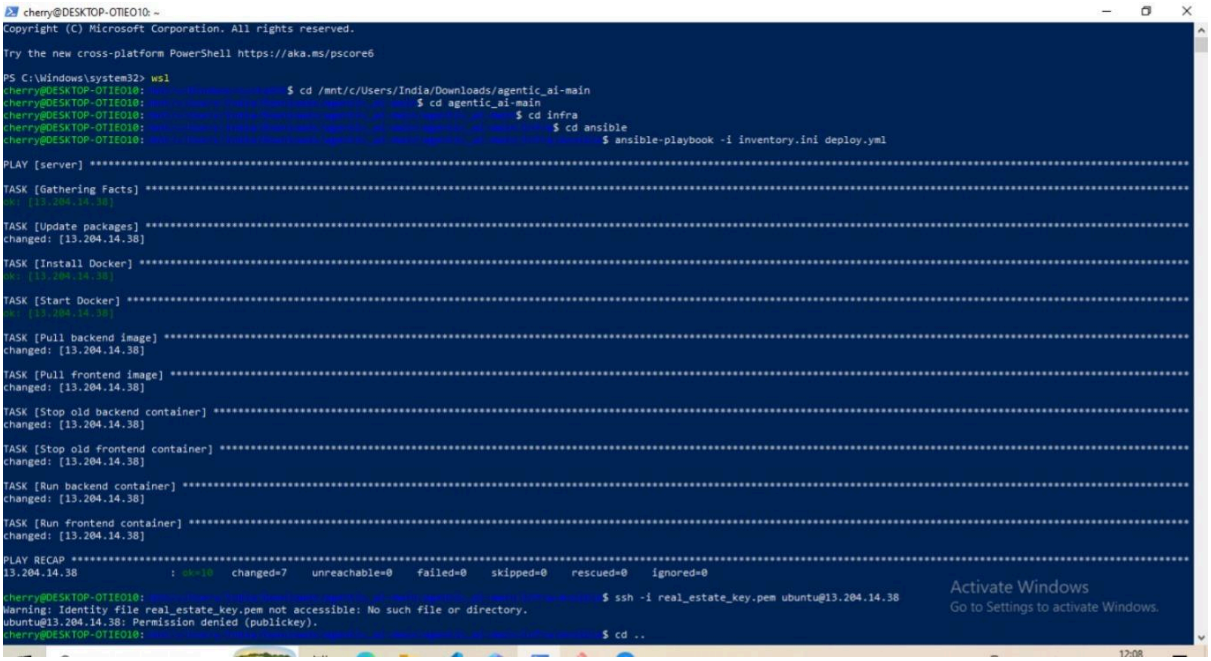
The outputs of the system can be categorized into three major components:

1. Screenshots / System outputs

The system produces automated and structured outputs at each stage of the DevOps pipeline, ensuring successful deployment and operation of the real estate application.

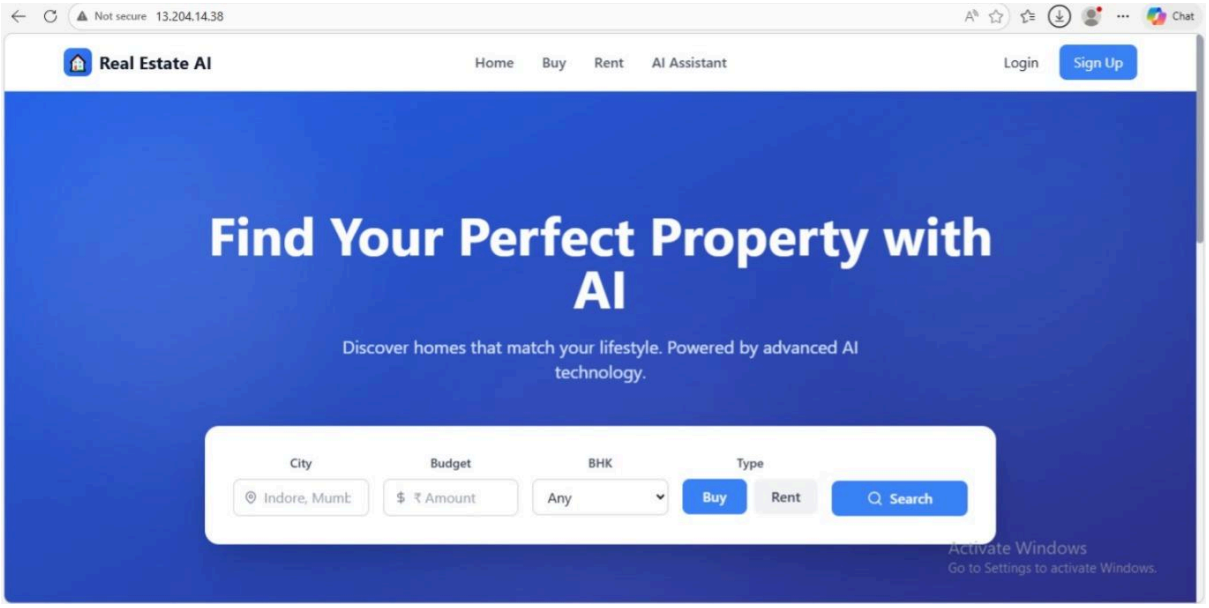


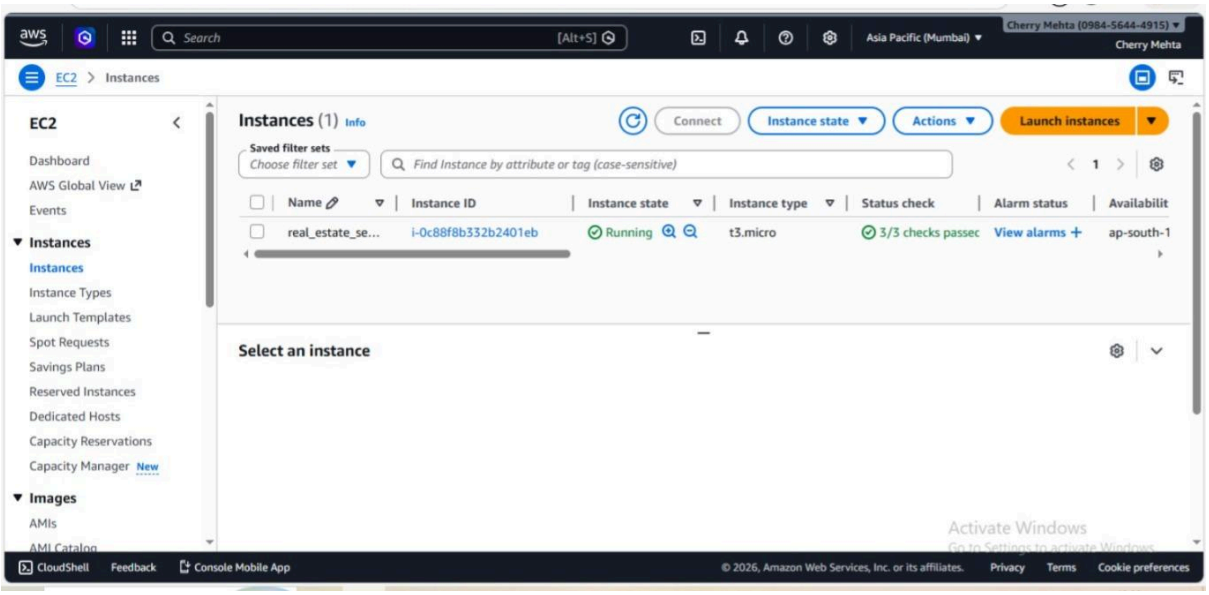


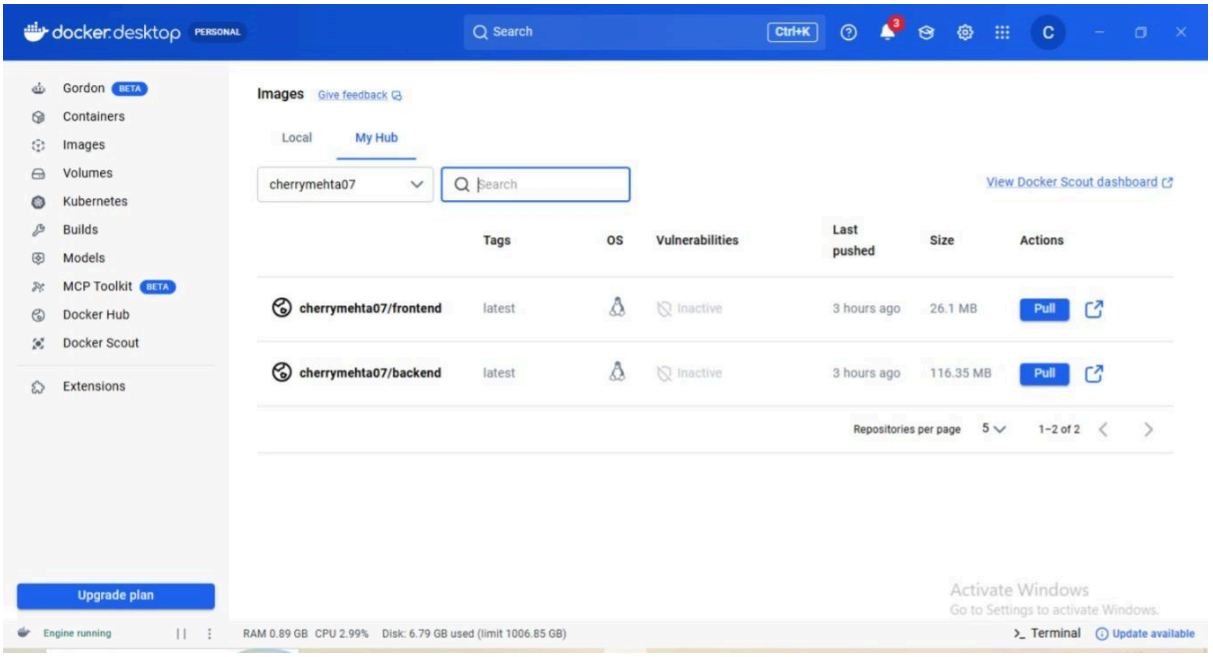


2.Reports / Dashboards / Models

The system generates various reports, dashboards, and operational outputs to monitor, manage, and analyze the deployment lifecycle and infrastructure performance.







Models Used:

1. Infrastructure Model

- Defined using Terraform
- Represents cloud resources such as EC2, VPC, IAM, and S3
- Written in HCL (HashiCorp Configuration Language)
- Ensures consistent and repeatable infrastructure provisioning

2. Configuration Model

- Defined using Ansible
- Represents server setup, software installation, and environment configuration
- Written in YAML
- Ensures automated and idempotent configuration

3. Containerization Model

- Implemented using Docker
- Represents the application and its dependencies packaged into containers
- Ensures portability across different environments

4. CI/CD Pipeline Model

- Implemented using GitHub Actions or Jenkins
- Represents automated workflow for build, test, and deployment
- Ensures continuous integration and continuous delivery

5. Deployment Model

- Combines all components (Terraform + Ansible + Docker + CI/CD)
- Represents the complete automated deployment lifecycle
- Ensures end-to-end automation from code commit to production

3. Key Outcomes

The major outcomes achieved through this project are as follow

1. **Successful Automation of Infrastructure**
   Infrastructure provisioning was fully automated using Terraform, eliminating manual setup.
2. **Consistent Environment Configuration**
   Servers were configured uniformly across all environments using Ansible.
3. **Containerized Application Deployment**
   The application was successfully packaged and deployed using Docker.
4. **End-to-End CI/CD Implementation**
   Automated pipelines using GitHub Actions / Jenkins enabled continuous integration and deployment.
5. **Reduced Deployment Time**
   Automation significantly decreased the time required for provisioning and deployment.
6. **Improved System Reliability**
   Reduced human errors and ensured stable and predictable deployments.

7. **Scalable System Architecture**
   The system supports scaling based on user demand and workload.

Conclusion

The Infrastructure as Code (IaC) Provisioning for Real Estate System successfully demonstrates the implementation of modern DevOps practices to automate infrastructure provisioning, configuration, and application deployment. The project effectively addresses the limitations of traditional manual deployment methods by introducing a streamlined, automated, and reliable workflow.

By integrating tools such as Terraform for infrastructure provisioning, Ansible for configuration management, Docker for containerization, and GitHub Actions / Jenkins for CI/CD automation, the system ensures consistency, scalability, and efficiency across all deployment stages.

The automated pipeline—from code commit to deployment—reduces human intervention, minimizes errors, and significantly improves deployment speed. Additionally, the use of cloud infrastructure provides flexibility, high availability, and the ability to scale resources based on demand.

Overall, the project achieves its objectives by delivering a robust, secure, and efficient deployment system. It highlights the importance of DevOps and IaC in modern software development and demonstrates how automation can enhance productivity, reliability, and system performance in real-world applications.

Future Scope & Enhancements

1. The current system provides a strong foundation for automated infrastructure provisioning and deployment using DevOps practices. However, several enhancements can be implemented in the future to further improve scalability, performance, and functionality.

2. **Kubernetes Integration for Orchestration**
   Integrate Kubernetes to manage and scale Docker containers efficiently in a clustered environment.
3. **Multi-Cloud Deployment Support**
   Extend the system to support multiple cloud platforms (AWS, Azure, GCP) for better flexibility and vendor independence.
4. **Advanced Monitoring and Alerting**
   Integrate tools like Prometheus and Grafana for real-time monitoring, alerting, and advanced analytics.
5. **Automated Security Scanning**
   Implement DevSecOps practices by integrating vulnerability scanning tools for containers and infrastructure.
6. **Auto-Scaling Optimization**
   Improve scaling policies to dynamically adjust resources based on real-time demand.
7. **Centralized Logging System**
   Integrate ELK Stack (Elasticsearch, Logstash, Kibana) for better log analysis and visualization.
8. **Database Integration**
   Add managed database services (like AWS RDS) for storing real estate application data.
9. **User Authentication & Authorization**
   Implement advanced security features such as OAuth and multi-factor authentication.